Procesos de Desarrollo de Software

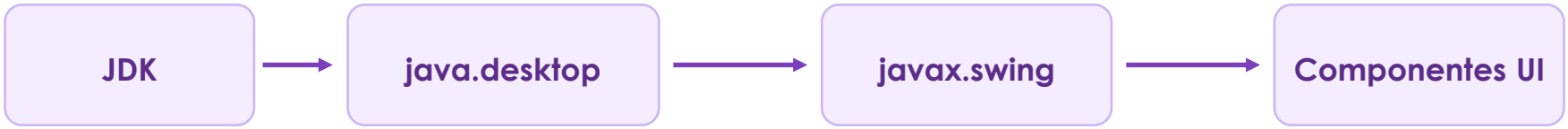
CLASE 2 ADICIONAL – SWING

Agenda de la clase

- SWING (repaso)

SWING

- Swing es una biblioteca gráfica para Java. Incluye widgets para interfaz gráfica de usuario tales como cajas de texto, botones, listas desplegables y tablas.
- Es un conjunto de componentes GUI extensibles para facilitar la vida del desarrollador y crear aplicaciones front-end / GUI basadas en JAVA. Se basa en la API AWT (Abstract Window Toolkit) y actúa como un reemplazo de la misma, ya que tiene casi todos los controles correspondientes a los controles AWT.



Swing

imports

```
1 import javax.swing.*;
2 import java.awt.*;
```

Swing

JFrame, JPanel, JButton, JLabel, JTable, JOptionPane.

AWT

Layouts, Color, Font, eventos base.

EDT

Hilo recomendado para crear/actualizar UI.

Controles

- **Elementos de la interfaz de usuario**: estos son los elementos visuales básicos que el usuario eventualmente ve e interactúa. Swing ofrece una gran lista de elementos comunes y ampliamente utilizados que varían de básicos a complejos.
- **Layouts**: definen cómo se deben organizar los elementos de la interfaz de usuario en la pantalla y proporcionan un aspecto final a la interfaz gráfica de usuario (GUI).
- **Comportamiento (Events)**: estos son los eventos que ocurren cuando el usuario interactúa con los elementos de la interfaz de usuario.

Elementos UI

- **Jlabel:** es un componente para colocar texto en un contenedor.
- **Jbutton:** Esta clase crea un botón etiquetado.
- **JColorChooser:** proporciona un panel de controles diseñados para permitir a un usuario manipular y seleccionar un color.
- **JCheckBox:** es un componente gráfico que puede estar en estado activado (verdadero) o desactivado (falso).
- **JRadioButton:** es un componente gráfico que puede estar en estado activado (verdadero) o desactivado (falso) dentro de un grupo.
- **Jlist:** presenta al usuario una lista de desplazamiento de elementos de texto.
- **JComboBox:** presenta al usuario un menú desplegable con opciones.
- **JTextField:** es un componente de texto que permite la edición de una sola línea de texto.
- **JPasswordField:** es un componente de texto diseñado para ingresar contraseñas.
- **JTextArea:** es un componente de texto que permite la edición de múltiples líneas de texto.
- **ImageIcon:** es una implementación de la interfaz Icon que pinta Iconos a partir de imágenes.
- **Jscrollbar:** representa un componente de barra de desplazamiento para permitir al usuario desplazarse en la ventana.
- **JOptionPane:** es un cuadro de diálogo estándar para solicitar algo al usuario o informarles de algo.
- **JFileChooser:** representa una ventana de diálogo desde la cual el usuario puede seleccionar un archivo.
- **JProgressBar:** sirve para mostrar un porcentaje de avance sobre la finalización de una tarea.

Layouts

- El diseño (layout) se refiere a la disposición de los componentes dentro del contenedor. De otra manera, podría decirse que el diseño coloca los componentes en una posición particular dentro del contenedor. La tarea de diseñar los controles se realiza automáticamente por el administrador de diseño (Layout Manager).

Layout Manager

- El administrador de diseño coloca automáticamente todos los componentes dentro del contenedor.
- Es posible colocar los controles a mano. Sin embargo, esto puede ser difícil puesto que.
  - Es muy tedioso manejar una gran cantidad de controles dentro del contenedor.
  - Por lo general, la información de ancho y alto de un componente no se da cuando tenemos que organizarlos.

Tipos de Layouts

- **BorderLayout**: la distribución de bordes organiza los componentes para que quepan en las cinco regiones: este, oeste, norte, sur y centro.
- **CardLayout**: trata cada componente en el contenedor como una tarjeta. Solo se puede ver una carta a la vez.
- **FlowLayout**: es el diseño predeterminado. Diseña los componentes en un flujo direccional.
- **GridLayout**: gestiona los componentes en forma de una cuadrícula rectangular.
- **GridBagLayout**: esta es la clase de administrador de diseño más flexible. El objeto de GridBagLayout alinea el componente verticalmente, horizontalmente o a lo largo de su línea base sin requerir que los componentes sean del mismo tamaño.
- **GroupLayout**: agrupa jerárquicamente los componentes para colocarlos en un contenedor.
- **SpringLayout**: coloca a los objetos de su contenedor asociado de acuerdo con un conjunto de restricciones.

Swing Containers

- Los contenedores son una parte integral de los componentes SWING GUI. Un contenedor proporciona un espacio donde se puede ubicar un componente.
- Las subclases de Contenedor se llaman Contenedor. Por ejemplo, JPanel, JFrame y JWindow.
- Un layout predeterminado está presente en cada contenedor (si bien este luego puede cambiarse por el método setLayout).

Swing Containers

- **Panel**

JPanel es el contenedor más simple. Proporciona espacio en el que se puede colocar cualquier otro componente, incluidos otros paneles.

- **Marco**

Un **JFrame** es una ventana de nivel superior con un título y un borde.

- **Ventana**

Un objeto **JWindow** es una ventana de nivel superior sin bordes y sin barra de menús.

Eventos

- El cambio en el estado de un objeto se conoce como Evento.
- Los eventos se generan como resultado de la interacción del usuario con los componentes de la interfaz gráfica (GUI). Por ejemplo:
  - hacer clic en un botón
  - mover el mouse
  - ingresar un carácter a través del teclado
  - seleccionar un elemento de la lista
  - desplazar la página

Tipos de Eventos

- **Eventos en primer plano (Foreground)**: estos eventos requieren la interacción directa del usuario. Se generan como consecuencias de una persona que interactúa con los componentes gráficos en la interfa (GUI). Por ejemplo: hacer clic en un botón, mover el mouse, ingresar un carácter a través del teclado, seleccionar un elemento de la lista, desplazar la página, etc.
- **Eventos de fondo (background)**: estos eventos requieren la interacción del usuario final. Las interrupciones del sistema operativo, la falla de hardware o software, el vencimiento del temporizador y la finalización de la operación son algunos ejemplos de eventos de fondo.

Manejadores de Eventos

- El manejo de eventos es el mecanismo que controla el evento y decide qué sucedería si ocurriera dicho evento. Este mecanismo tiene un código que se conoce como controlador de eventos, que se ejecuta cuando ocurre un evento.
- Java usa el modelo de eventos de delegación (**Delegation Event Model**) para manejar los eventos. Este modelo define el mecanismo estándar para generar y manejar los eventos.

Manejadores de Eventos

- El modelo de evento de delegación tiene lo siguiente:
  - **Source**: la fuente es un objeto sobre el cual ocurre el evento. La fuente es responsable de proporcionar información del evento ocurrido a su controlador. Java nos proporciona clases para el objeto fuente.
  - **Listener**: también se lo conoce como controlador de eventos (event handler). El listener es responsable de generar una respuesta a un evento. Desde el punto de vista de la implementación de Java, el listener también es un objeto. El listener espera hasta recibir un evento. Una vez que se recibe el evento, el listener procesa el evento y luego regresa.
- En este modelo, el listener debe estar registrado con el objeto fuente para que pueda recibir la notificación del evento. Esta es una forma eficiente de manejar el evento porque las notificaciones de eventos se envían solo a aquellos listeners que desean recibirlas.

Manejadores de Eventos

Paso 1: el usuario hace clic en el botón y se genera el evento.

Paso 2: el objeto de la clase de evento en cuestión se crea automáticamente y la información sobre el origen y el evento se llena en el mismo objeto.

Paso 3: el objeto del evento se reenvía al método de la clase de listener registrada.

Paso 4: el método se ejecuta.

Swing Magic Cafe

module-info.java

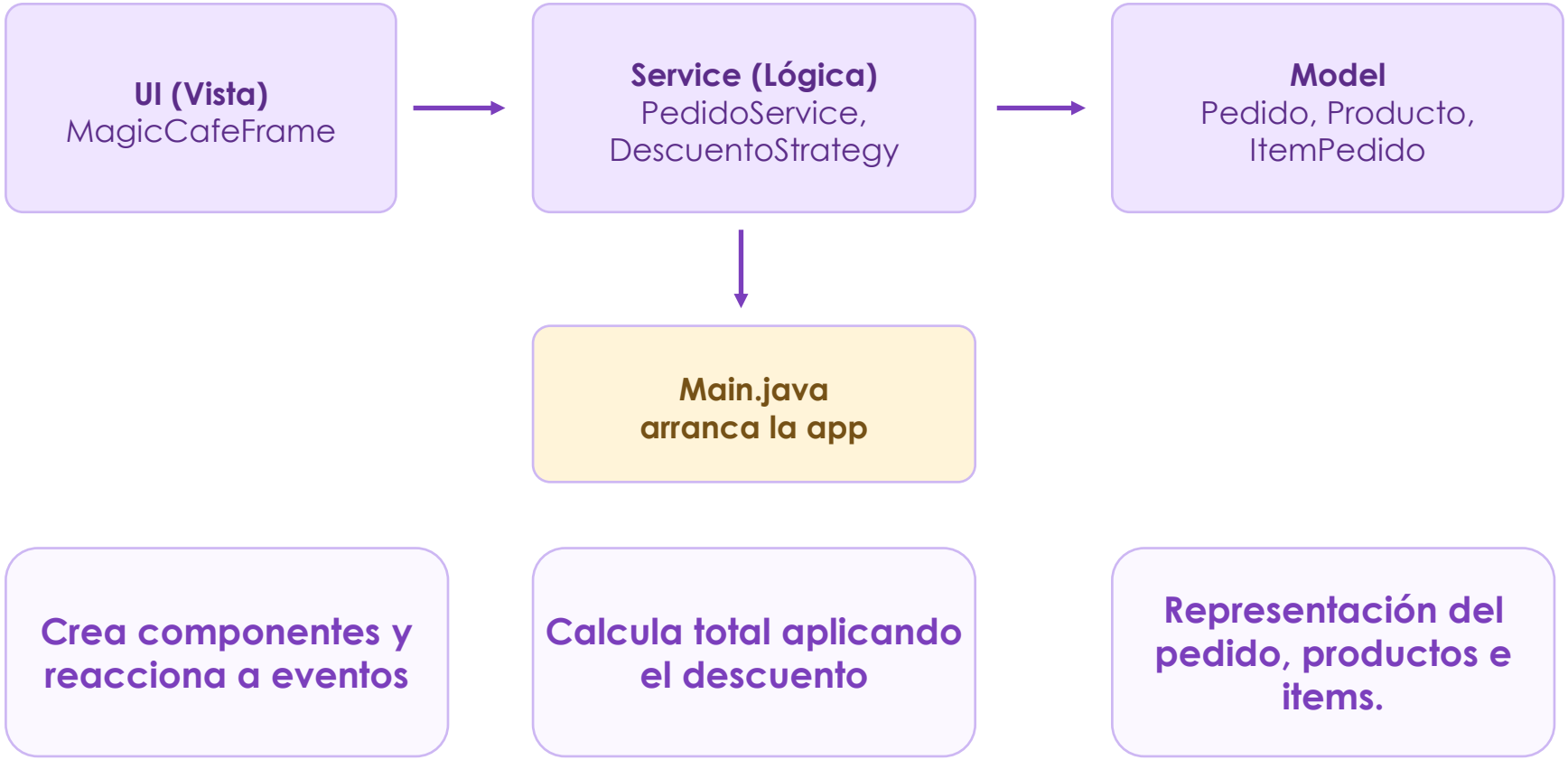
```
module POO_Clase_8_Cafe {
    requires java.desktop;
}
```

estructura

```
src/main/java
 └ com.uade.swingmagiccafe
     ├ Main.java
     ├ model
     ├ service
     └ ui/MagicCafeFrame.java
```

- Si faltan JFrame, JButton, JComboBox o JTable: revisar imports y java.desktop.
- Si usan WindowBuilder: abrir MagicCafeFrame.java con “Open With > WindowBuilder Editor”.

Arquitectura



Paso 1: Main.java

- El punto de entrada crea la ventana dentro del Event Dispatch Thread.

Main.java

```java
public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            MagicCafeFrame frame = new MagicCafeFrame();
            frame.setVisible(true);
        });
    }
}
```

📜 ¿Por qué invokeLater?

Le pedimos a Swing que cree y modifique la interfaz desde su hilo de UI.

- main no dibuja la pantalla: solo instancia el JFrame.
- setVisible(true) muestra la ventana.
- Esto es una buena práctica para evitar problemas de concurrencia en Swing.

Paso 2: La ventana principal

- MagicCafeFrame hereda de JFrame y concentra la pantalla.

MagicCafeFrame.java

```java
public class MagicCafeFrame extends JFrame {

    private final Pedido pedido = new Pedido();
    private final PedidoService pedidoService = new
PedidoService();

    private final JComboBox<Producto> productosCombo;
    private final JTextField cantidadField;
    private final JTable tablaPedido;
    private final JLabel totalLabel;
}
```

🪟 JFrame

La ventana principal de la app.

🧩 Campos UI

Se guardan como atributos porque luego se actualizan.

📦 Pedido + Service

La pantalla usa el modelo y delega cálculo de total.

Paso 3: Constructor de la pantalla

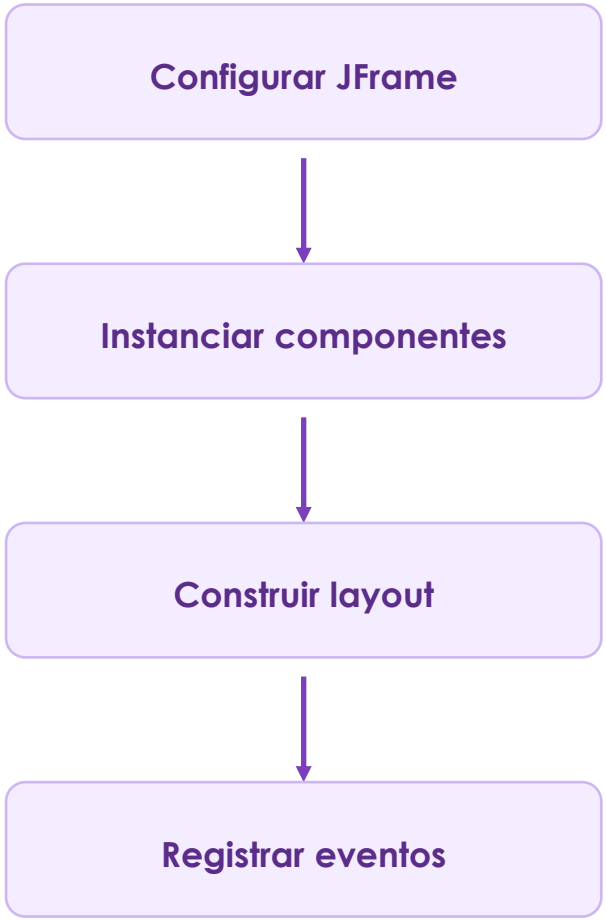
- Primero configuramos la ventana, luego creamos componentes y eventos.

Constructor

```java
public MagicCafeFrame() {
    setTitle("Swing Magic Cafe ☕✨");
    setSize(850, 550);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(EXIT_ON_CLOSE);

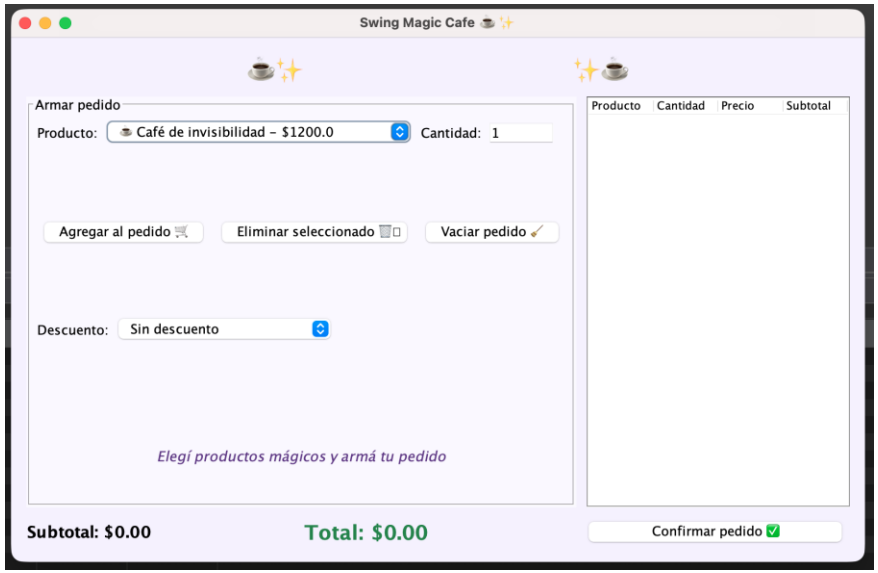
    productosCombo = new JComboBox<>(crearProductos());
    cantidadField = new JTextField("1", 5);
    descuentosCombo = new
JComboBox<>(crearDescuentos());

    construirPantalla();
    registrarEventos();
}
```



Paso 4: Layouts: ubicar componentes

- Swing no “flota” los elementos: usamos Layout Managers.



BorderLayout

```
1 JPanel root = new JPanel(new BorderLayout(10, 10));
2 root.add(titulo, BorderLayout.NORTH);
3 root.add(panelFormulario, BorderLayout.WEST);
4 root.add(new JScrollPane(tablaPedido), BorderLayout.CENTER);
5 root.add(panelTotales, BorderLayout.SOUTH);
```

⬆️ **NORTH**

Título

⬅️ **WEST**

Formulario

🎯 **CENTER**

Tabla

Paso 5: Componentes principales

- Cada widget de la pantalla es un objeto Java.

widgets

```
productosCombo = new JComboBox<>(crearProductos());
cantidadField = new JTextField("1", 5);

JButton agregarButton = new JButton("Agregar al pedido 🛒");
JButton eliminarButton = new JButton("Eliminar seleccionado 🗑️");
JButton vaciarButton = new JButton("Vaciar pedido 🧹");
```

📋 **JComboBox<Producto>**
Lista desplegable de productos.

⌨️ **JTextField**
Entrada de cantidad.

🔘 **JButton**
Dispara una acción al hacer click.

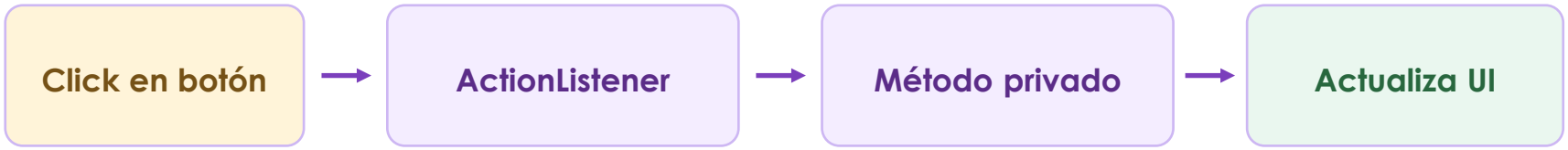
🏷️ **JLabel**
Muestra mensajes, subtotal y total.

Paso 6: Eventos: del click al método

- El usuario hace click; Swing ejecuta un listener.

listeners

```
1 agregarButton.addActionListener(e -> agregarProducto());
2 eliminarButton.addActionListener(e -> eliminarSeleccionado());
3 vaciarButton.addActionListener(e -> vaciarPedido());
4 confirmarButton.addActionListener(e -> confirmarPedido());
```



Paso 7: Agregar producto

- Validamos la entrada, actualizamos el modelo y refrescamos la pantalla.

agregarProducto

```java
private void agregarProducto() {
    try {
        Producto producto = (Producto) productosCombo.getSelectedItem();
        int cantidad = Integer.parseInt(cantidadField.getText());

        if (cantidad <= 0) {
            JOptionPane.showMessageDialog(this,
                "La cantidad debe ser mayor a cero.");
            return;
        }

        pedido.agregarItem(producto, cantidad);
        actualizarTablaYTotales();
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this,
            "Ingresá una cantidad numérica válida.");
    }
}
```

1. Leer UI

Producto seleccionado y texto de cantidad.

2. Validar

Cantidad numérica y mayor a cero.

3. Actualizar

Agregar ítem al pedido y refrescar tabla.

Paso 8: JTable y DefaultTableModel

- La tabla se llena desde el pedido cada vez que cambia algo.

crear tabla

```
tableModel = new DefaultTableModel(
    new Object[]{"Producto", "Cantidad", "Precio", "Subtotal"}, 0
) {
    @Override
    public boolean isCellEditable(int row, int column) {
        return false;
    }
};

tablaPedido = new JTable(tableModel);
```

refrescar tabla

```
tableModel.setRowCount(0);

for (ItemPedido item : pedido.getItems()) {
    tableModel.addRow(new Object[]{
        item.getProducto().getNombre(),
        item.getCantidad(),
        item.getProducto().getPrecio(),
        item.getSubtotal()
    });
}
```

- La JTable muestra datos; el DefaultTableModel es la fuente de esos datos.
- setRowCount(0) limpia la tabla antes de reconstruirla.

Paso 9: Modelo de dominio

- La lógica del pedido vive fuera de la interfaz.

Producto + ItemPedido

```java
public class Producto {
    private final String nombre;
    private final double precio;
    private final String emoji;
}

public class ItemPedido {
    private final Producto producto;
    private final int cantidad;

    public double getSubtotal() {
        return producto.getPrecio() * cantidad;
    }
}
```

Pedido

```java
public class Pedido {
    private final List<ItemPedido> items = new ArrayList<>();

    public void agregarItem(Producto producto, int cantidad) {
        items.add(new ItemPedido(producto, cantidad));
    }

    public double calcularSubtotal() {
        return items.stream()
            .mapToDouble(ItemPedido::getSubtotal)
            .sum();
    }
}
```

Paso 10: Strategy para descuentos

- Diferentes formas de calcular el total sin llenar la UI de if/else.

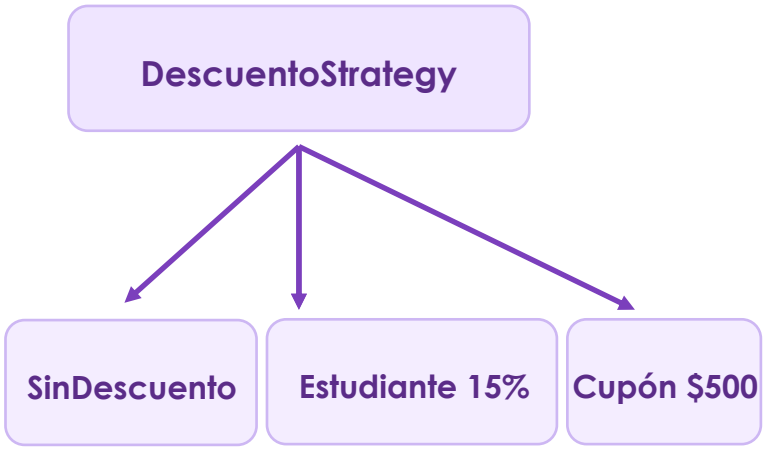
Strategy

```java
public interface DescuentoStrategy {
    String getNombre();
    double aplicar(double subtotal);
}

public class DescuentoEstudiante implements DescuentoStrategy {
    public double aplicar(double subtotal) {
        return subtotal * 0.85;
    }
}
```

PedidoService

```java
public class PedidoService {
    public double calcularTotal(Pedido pedido,
                                DescuentoStrategy descuento) {
        return descuento.aplicar(pedido.calcularSubtotal());
    }
}
```



Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

